SOFTWARE AND PROGRAMMING ▪ SIAM NEWS PROPOSAL

# Who Counts as a Neighbor?

How ProximityGraphs turns one point cloud into many mathematical worlds

Héctor Maravillo Diego Villarreal de la Cerda Heriberto Espino-Montelongo

Proposed byline; affiliations, author order, and biographies require contributor approval.

A planner studying a map of water stations wants to know which station serves each home. An ecologist wants to trace the outer boundary of scattered observations. A network engineer wants to decide which sensors might reasonably be connected. All three begin with points in the plane, but their questions require different meanings of *neighbor*.

That distinction matters. Distance alone does not tell us which connections should exist: a pair of points may share a nearest-site boundary, belong to the same geometric envelope, or qualify as neighbors because a particular region between them contains no other point. Each choice produces a different mathematical structure—and potentially a different scientific interpretation.

ProximityGraphs is an open-source Python library that makes these choices explicit. It brings together classic constructions such as Voronoi and Delaunay, Gabriel and relative neighborhood graphs, beta-skeletons, alpha-hulls, nearest-neighbor graphs, and related families under a common point-set workflow. Instead of coding every geometric rule from scratch, users can build, visualize, and compare alternatives on the same data.

## A city divided by distance

Consider a hypothetical city with temporary drinking-water stations. If people always visit their closest station *as the crow flies*, each station occupies a Voronoi cell: the set of locations closer to it than to any other station. Formally, for sites $P = \{p_1, \ldots, p_n\}$, the cell of site $p_i$ is

$$V_i = \{x : \|x - p_i\| \leq \|x - p_j\| \text{ for every } j\}.$$

Connecting sites whose Voronoi cells share a boundary gives the Delaunay dual. The first object answers a coverage question; the second reveals which service territories meet. Both emerge from the same coordinates.

There is a compelling historical connection. During London's 1854 cholera outbreak, physician John Snow investigated the geographic relationship between deaths and water pumps. Later spatial analyses revisited nearest-pump catchments. Our diagram is **not** Snow's original map, and straight-line Voronoi regions are not a substitute for realistic travel-time, capacity, or demand models. They are a transparent starting point for explaining spatial allocation.

**Figure 1 · Water stations: Voronoi and Delaunay**

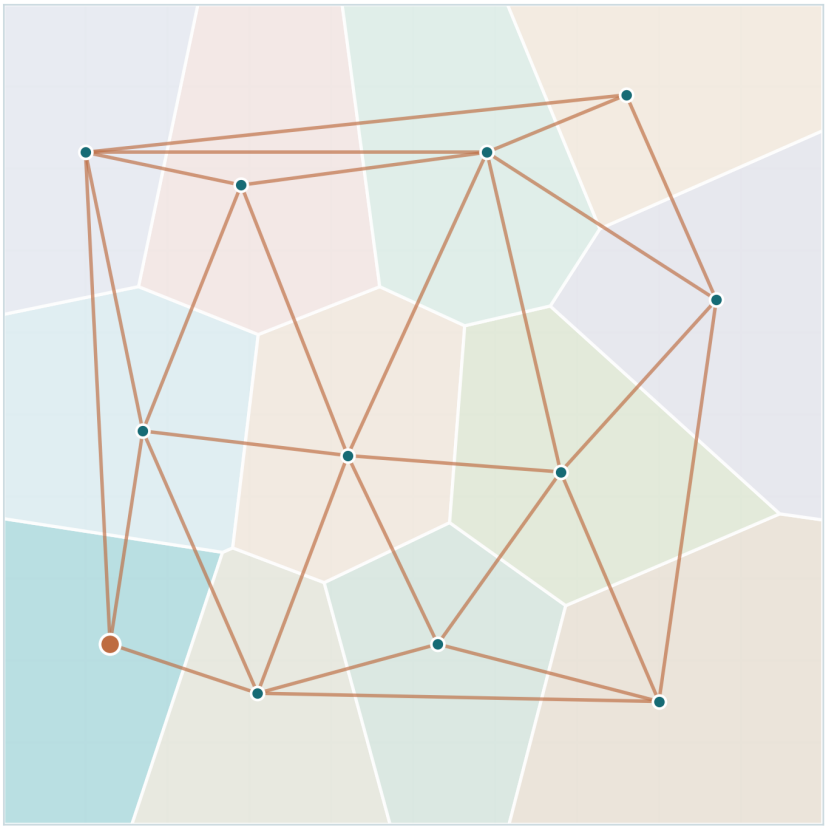


**Figure 1:** Hypothetical water stations with Euclidean Voronoi service cells and Delaunay-adjacency edges. Select or move a station to see which regions change. The library constructs the site-adjacency graph; finite Voronoi cells are clipped separately. Explore the interactive figure. Original static geometry and image courtesy of the authors; all sites are synthetic.

## A boundary made from circles

Now imagine ecological observations distributed around an irregular landscape. The familiar convex hull—like a rubber band stretched around the outermost points—ignores bays and indentations. It may enclose regions with no observations and conceal the outline's geometry.

The alpha-shape family introduces a scale into boundary construction. Its circular-arc relative, the *alpha-hull*, can trace a boundary with arcs rather than straight segments. In the ProximityGraphs convention used in our example, an arc's supporting circle has radius $R = 1/|\alpha|$, with a negative alpha parameter. Changing this radius changes which concavities become visible.

The second visualization makes those circles tangible: select an arc to reveal its supporting circle, or move an observation and watch the envelope change. The outcome is a **geometric description of sampled points**, not a statistical estimate of habitat occupancy. Whether an outline represents a real ecological boundary depends on survey design and additional modeling.

**Figure 2 · Habitat observations: circular alpha-hull**

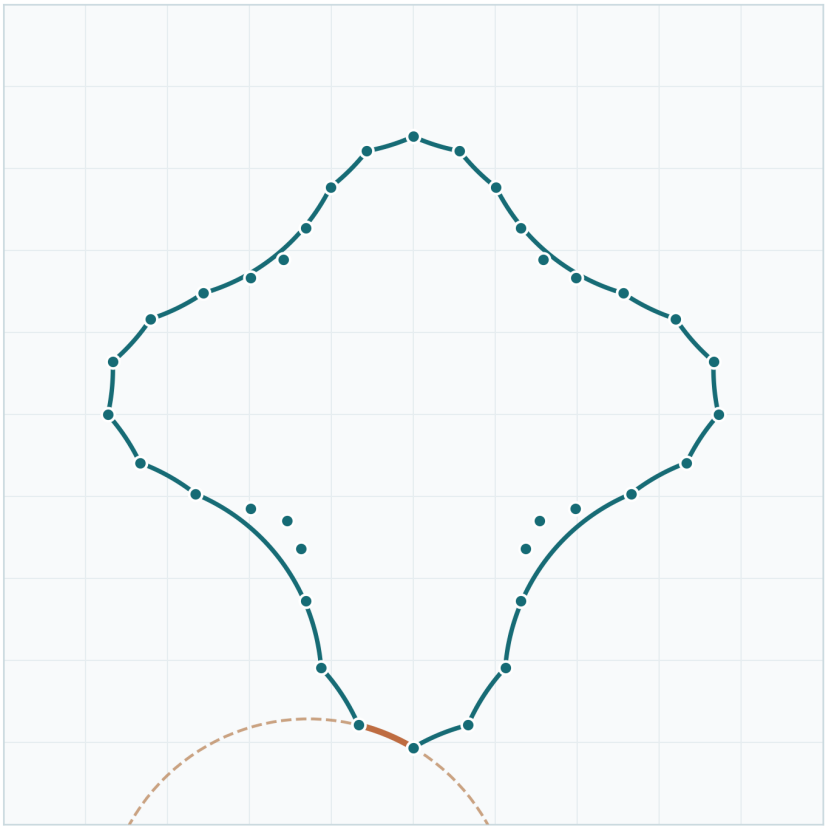


**Figure 2:** Alpha-hull arcs around synthetic habitat observations, highlighting one supporting circle. The interactive version lets readers adjust $\alpha$ and the point positions; original static arcs come from ProximityGraphs. Explore the interactive figure. Original static geometry and image courtesy of the authors; all sites are synthetic.

## When an empty region creates an edge

A third problem arises when points represent wireless sensors. Connecting every pair yields a dense network that says little about local structure. Connecting only the nearest pairs imposes a different rule. A *beta-skeleton* provides an interpretable alternative: connect two sites only when a region determined by that pair is empty of other sites.

In the lune-based family, the relevant region is a lens formed by two overlapping disks. The parameter $\beta$ controls its shape. At $\beta = 1$, the construction gives the Gabriel graph; at $\beta = 2$, the relative neighborhood graph, under standard general-position conventions. A point added inside the relevant lens can eliminate a previously permissible edge.

That local cause-and-effect is the focus of the third visualization. Readers can change $\beta$, inspect a candidate edge, and see the two disks whose intersection must be empty. The graph identifies *geometric candidates*, not guaranteed radio connections: physical deployment also requires propagation, terrain, power, and reliability models.

**Figure 3 · Sensor candidate links: beta-skeleton**

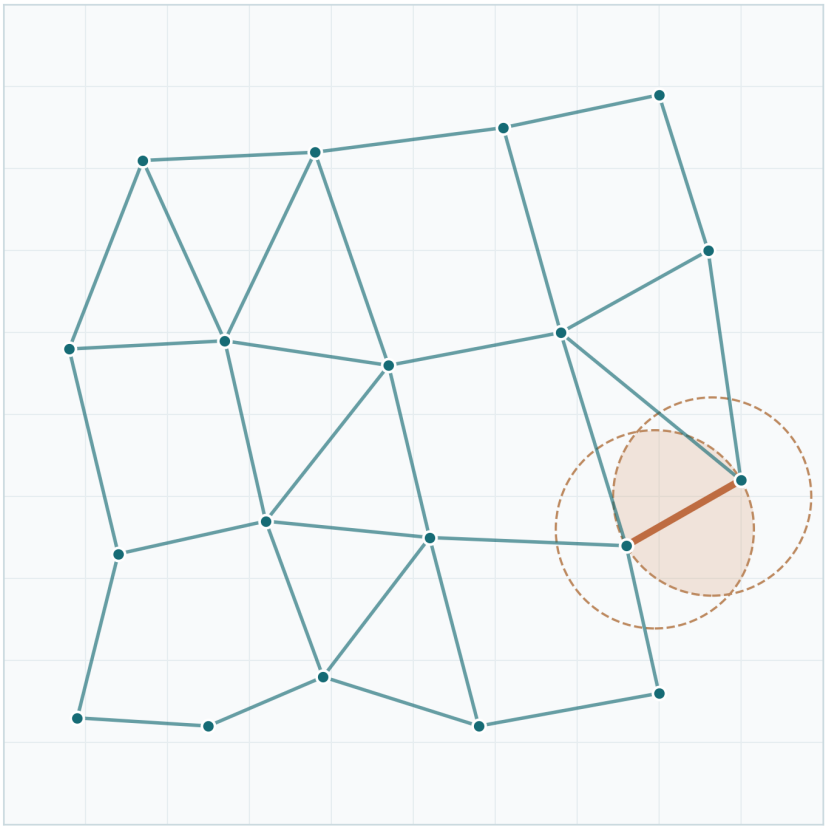


**Figure 3:** A beta-skeleton over hypothetical sensor locations with the selected edge's empty lens shown. Moving or adding a point can change the edge set. The static graph was generated with ProximityGraphs. Explore the interactive figure. Original static geometry and image courtesy of the authors; all sites are synthetic.

## One point set, several answers

These examples suggest a practical question for researchers: **What changes when we change the neighborhood rule but keep the data fixed?** ProximityGraphs is designed for precisely that comparison. Its point-set objects, graph constructors, visualization tools, and experiment helpers create a workflow in which a hypothesis about spatial structure can be tested rather than merely illustrated.

A small Python example captures the idea:

```
import proximitygraphs as pg
points = pg.SetPoints.grid(shape=(3, 3))
gabriel = pg.GG(points)
relative = pg.RNG(points)
delaunay = pg.DelaunayG(points)
```

The same nine sites produce three graph constructions that encode different definitions of adjacency. The library also provides utilities for repeated experiments and optional geospatial-data integration. It complements general graph and scientific-computing packages by focusing on the geometry that determines which edges exist.

To let readers explore the distinctions without installing Python, we built an interactive companion. Its original examples use geometry exported by ProximityGraphs; when readers add, delete, or drag points,

independent browser code recomputes the demonstrations and labels them accordingly. The source, figures, and generation scripts are public and reproducible.

**A graph begins with a definition, not a drawing.** Before treating any line as a road, relationship, or possible communication link, ask which rule created it—and which scientific assumptions make that rule useful. ProximityGraphs provides a place to ask the question with code, and the interactive diagrams let anyone see the answer change.

**Explore:** Source and Python package ▪ Installation ▪ Interactive figures. **Install:** `pip install proximitygraphs`. (MIT License.)

## References


[1] Edelsbrunner, H., Kirkpatrick, D. G., & Seidel, R. (1983). On the shape of a set of points in the plane. *IEEE Transactions on Information Theory, 29*(4), 551–559.

[2] Gabriel, K. R., & Sokal, R. R. (1969). A new statistical approach to geographic variation analysis. *Systematic Zoology, 18*(3), 259–278.

[3] Maravillo, H., Villarreal de la Cerda, D., & Espino-Montelongo, H. (2026). *ProximityGraphs* [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.21088183.

[4] Snow, J. (1855). *On the mode of communication of cholera* (2nd ed.). John Churchill.

[5] Toussaint, G. T. (1980). The relative neighbourhood graph of a finite planar set. *Pattern Recognition, 12*(4), 261–268.


## About the authors


**Héctor Maravillo** develops mathematical and computational research involving proximity graphs. *Confirm affiliation and biography.*

**Diego Villarreal de la Cerda** contributes to ProximityGraphs and its computational development. *Confirm affiliation and biography.*

**Heriberto Espino-Montelongo** studies actuarial science and data science at Universidad de las Américas Puebla and works on geometric and computational methods. *Confirm preferred biography.*